Dokumentasi Sistem PR Shuttle

Buku Panduan Teknis & Operasional Terintegrasi

Sistem Manajemen Tiket, Finance Operasional, & Data Warehouse

Disusun Secara Otomatis Berdasarkan Arsitektur NestJS & MongoDB

Contents

1 Ringkasan Sistem **2**

2 1. Alur Data Aplikasi (Data Flow) **2**
2.1 1.a Booking oleh CS Loket (CS / Reservasi) 2
2.2 1.b Booking oleh OTA External 2
2.3 1.c Siklus Finance Inti (Uang Saku → Realisasi → Posting Ledger) 2
2.4 1.d Persiapan Jadwal oleh Admin Operasional 3
2.5 1.e Pembuatan Laporan oleh Manajemen 3

3 2. Ketergantungan Antar Modul (Module Dependencies di .module & .dto) **4**

4 3. Perilaku Pengguna Per Peran **5**
4.1 3.1 Superadmin / Manajemen 5
4.2 3.2 Admin Operasional 5
4.3 3.3 Customer Service (CS) / Kasir Loket 5
4.4 3.4 Kasir Garasi (CASHIER) 5
4.5 3.5 Finance Pusat (FINANCE) 5

5 4. Flowchart Per Alur Bisnis **6**
5.1 4.a Full Booking Flow 6
5.2 4.b OTA Booking Flow (API) 7
5.3 4.c Finance Flow (Termasuk Overwrite, Manifest, & Premi) 8
5.4 4.d Schedule Preparation Flow 9
5.5 4.e Seat Locking Race Condition Prevention 10
5.6 4.f Order Expiry BullMQ Worker Flow 11

6 5. Use Case Diagram & Deskripsi **12**

Ringkasan Sistem

PR Shuttle adalah backend aplikasi shuttle bus berbasis NestJS dengan MongoDB sebagai *primary database*, Redis untuk *distributed lock* dan *transient state*, serta BullMQ untuk *asynchronous job processing*.

Secara bisnis, sistem ini dibagi ke lima domain utama:

1. Penjualan tiket dan reservasi lintas outlet.
2. Operasional keberangkatan dan manifest perjalanan.
3. **Keuangan operasional driver** (Flow ini dikontrol sangat ketat, melibatkan *Uang Saku*, overwrite *Mastercost*, *Realisasi*, *Premi*, *Ledger*, dan baru dapat berjalan setelah Manifest **Dicetak**).
4. Integrasi eksternal OTA (*Online Travel Agent*) dan payment workflow.
5. Reporting dan dashboard manajemen berbasis *Data Warehouse* (DW).

1. Alur Data Aplikasi (Data Flow)

1.a Booking oleh CS Loket (CS / Reservasi)

- CS Loket membuka menu **Reservasi / Cari Tiket** → mengisi tanggal keberangkatan + outlet asal/tujuan.
- Sistem mencari `Departure` dengan status `RELEASE` dan aktif → menampilkan daftar jadwal & seat map.
- CS klik kursi → Redis membuat temporary lock (`SET NX EX 3600`). Seat ditandai tidak tersedia untuk outlet lain.
- CS mengisi data penumpang → Sistem menghitung harga akhir (base rate, dynamic pricing, tuslah, promo).
- CS memilih payment method (`TUNAI`, `DP`, `EDC`, dsb) → Klik **Pesan / Cetak Tiket**.
- Sistem membuat `Order` (Status menjadi `PAID` / `DP_PAID`) → membuat `Ticket`.
- Kursi ditandai *occupied* permanen di `DepartureAvailability` → Sinkronisasi DW async → Tiket dicetak.

1.b Booking oleh OTA External

- OTA memanggil `POST /integration/{ota}/booking` bertamengkan `ApiKeyThrottlerGuard`.
- Sistem memeriksa seat → `LockSeatService` dijalankan → `Order` dibuat (`PENDING`).
- Permament lock sementara ditaruh → BullMQ enqueues `order-expired-queue` (1 jam).
- OTA memanggil `issuebooking` → tiket terbit, order `PAID`. (Jika kadaluwarsa, queue BullMQ mencabut reservasi otomatis).

1.c Siklus Finance Inti (Uang Saku → Realisasi → Posting Ledger)

Catatan Penting: Finance flow HANYA bisa berjalan jika keberangkatan sudah memiliki manifest yang tercetak secara sistem.

- **Admin Operasional Cetak Manifest**: Sebagai syarat awal absah. Manifest dicetak → Flow operasional keuangan terbuka.
- **Uang Saku (Overwriting Mastercost)**: Kasir Garasi menu Uang Saku → memasukkan BBM, Parkir, Dll. **Tindakan ini sangat krusial**: Jika nominal uang saku berbeda dari standar Mastercost, maka nilai Uang Saku tersebut **AKAN MENGOVERWRITE** (Jatuh tumpang tindih) nilai Mastercost di dalam `Departure Manifest`.
- **Manifest Menjadi Source of Truth**: Sejak di-overwrite, entitas Manifest resmi menjadi acuan data tunggal. Ledger menciptakan KREDIT via source `UANG_SAKU`.
- **Realisasi**: Supir kembali membawa struk. Kasir Garasi menginput pengeluaran riil di "Realisasi Keberangkatan". Sistem **secara otomatis menarik value** yang sudah di-overwrite dari Manifest tadi.
- **Posting Final**: Finance Pusat mengevaluasi draft. Saat di-klik **Posting Final**, Ledger permanen terbuat (`DEBIT` vs `CREDIT`), saldo ter-rebuild.
- **Pemrosesan Premi**: Sistem lalu mencairkan hak *Premi* (*Bonus Driver/Crew*), yang kemudian langsung dipotongkan validasinya menembus Ledger.

1.d Persiapan Jadwal oleh Admin Operasional

- Buka "Dashboard Penjadwalan" → Create scheduler bernama (Misal "Lebaran 2") → Generate 90 hari keberangkatan asinkron.
- Buka "Jadwal Bus Harian" → Pilih departure spesifik → Assign Driver, Shuttle, Master Cost.
- Klik **Release** → `Departure` ganti remark `RELEASE` → Terlihat oleh CS Loket untuk dibooking.

1.e Pembuatan Laporan oleh Manajemen

- Tiap order `PAID` berlabuh → trigger async mensinkronisasi ke Tabel **Data Warehouse (DW)**.
- Management membuka filter tanggal di dashboard → Server men-query tabel DW tanpa mengganggu tabel Order live yang sibuk → Return *Pie charts* & *Bar charts*.

2. Ketergantungan Antar Modul (Module Dependencies di .module & .dto)

Berdasarkan standar NestJS, dependensi antar entitas secara eksplisit dipetakan di file `*.module.ts` melalui `imports[]` dan di `*.dto.ts` melalui *class-validator* relasional.

- **DepartureModule**:
  - *Dependensi File (`.module`)*: Mengimpor `RouteModule`, `ShuttleModule`, `OutletModule`.
  - *Data Transit (`.dto`)*: `CreateDepartureDto` mewajibkan pengikatan `routeId`, `shuttleId`, dan men-trigger Job sinkronisasi ke `DepartureAvailability`.
- **OrderModule**:
  - *Dependensi*: `DepartureModule`, `PromoModule`, `VoucherModule`, `PaymentModule`. Emit: `order-expired-queue`.
  - *DTO*: `CreateOrderDto` memvalidasi secara utuh `departureId` dan *lock validation*.
- **FinanceModule (UangSaku & Realisasi)**:
  - *Dependensi*: Berpusat berat pada **DepartureManifestModule** dan **LedgerModule**.
  - *Aturan Krusial DTO*: DTO di sini dirancang untuk **menimpa** (overwrite) atribut biaya (BBM, parkir, dsb) langsung menembus `DepartureManifest` yang beralih rupa menjadi *Source of Truth*.
- **IntegrationModule (OTA)**:
  - Di dalam `IntegrationModule` terikat `ApiKeyThrottlerGuard` untuk membatasi traffic pihak ketiga (Traveloka dkk.). Ia membaca ketersediaan dari `DepartureAvailabilityModule`.

3. Perilaku Pengguna Per Peran

3.1 Superadmin / Manajemen

- **Halaman Akses**: Dashboard Utama, Master Data, Laporan DW, Kas Harian.
- **Aksi & Proses**: Menekan Master Rute dan menekan tombol *Simpan*, NestJS melancarkan `sync-departure-update-route-queue`. Management menekan filter Laporan DW dan menerima analitik langsung tanpa mengganggu database Order riil.

3.2 Admin Operasional

- **Halaman Akses**: Dashboard Penjadwalan, Jadwal Bus Harian, Departure Manifest.
- **Aksi & Proses**: Menekan *Generate Jadwal*, BullMQ mencetak 90 baris data `Departure`. Admin menekan tombol *Cetak Manifest*, ini adalah syarat prasyarat utama sebelum Kasir Garasi boleh menjalankan roda form Uang Saku.

3.3 Customer Service (CS) / Kasir Loket

- **Halaman Akses**: Reservasi, Form Pemesanan, Setoran Outlet.
- **Aksi & Proses**: Memilih Bangku → Menghantam API Redis Lock. Mengeklik Pembayaran `TUNAI` → Memutar status jadi `PAID` dan tiket keluar.

3.4 Kasir Garasi (CASHIER)

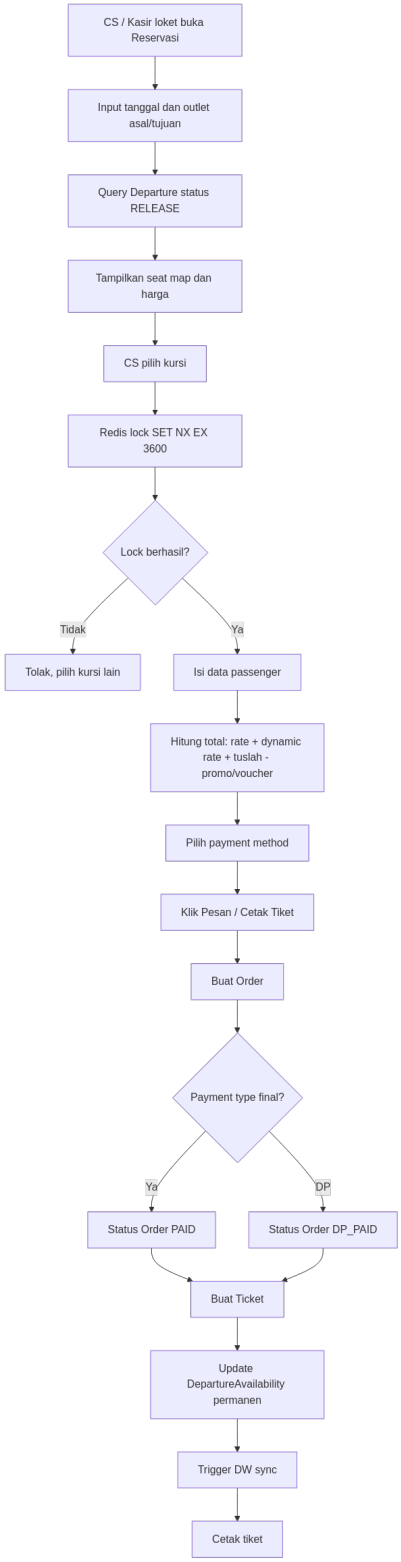
- **Halaman Akses**: Uang Saku, Realisasi Keberangkatan.
- **Aksi & Proses**: Menyuntik *Uang Saku Draf*, yang jika nominalnya membelot dari pakem *Mastercost*, otomatis meniban aslinya di `Departure Manifest`. Setelah supir pulang, Kasir menekan draf riil "Realisasi" di mana parameter diambil mutlak dari manifest tersebut.

3.5 Finance Pusat (FINANCE)

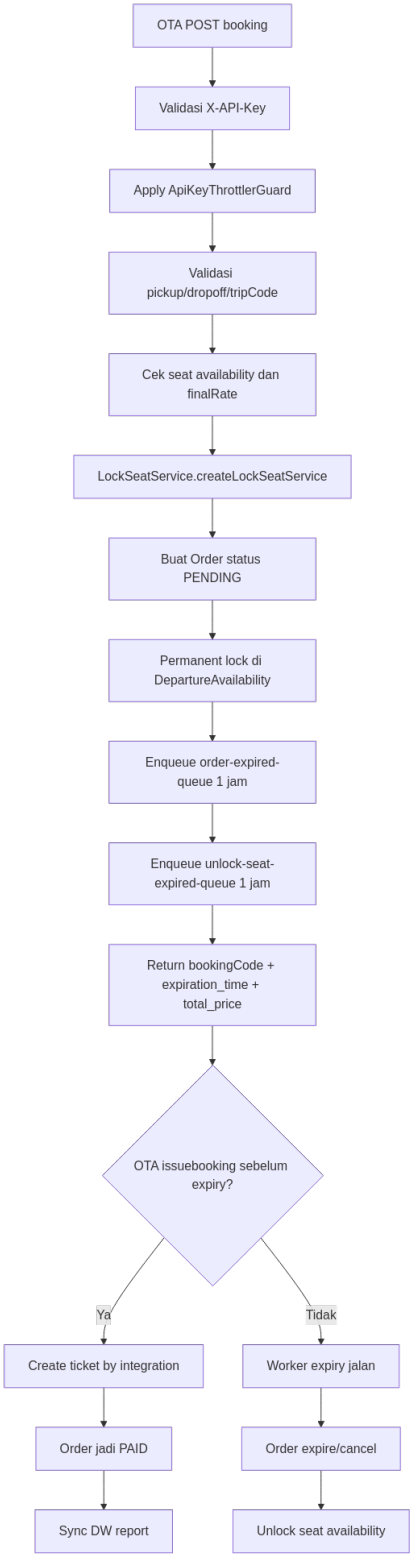
- **Halaman Akses**: Buku Besar Ledger, Review Realisasi, **Premi Driver**.
- **Aksi & Proses**: Menyaksikan gambar draf Realisasi. Menggebrak klik **"Posting Final"** → Buku Ledger mencatat utang piutang. Finance mengoperasikan halaman **Premi Driver/Crew** dengan mengunci *claim flag* dan melontarkan ledger potong klaim.

4. Flowchart Per Alur Bisnis

4.a Full Booking Flow



4.b OTA Booking Flow (API)



4.c Finance Flow (Termasuk Overwrite, Manifest, & Premi)

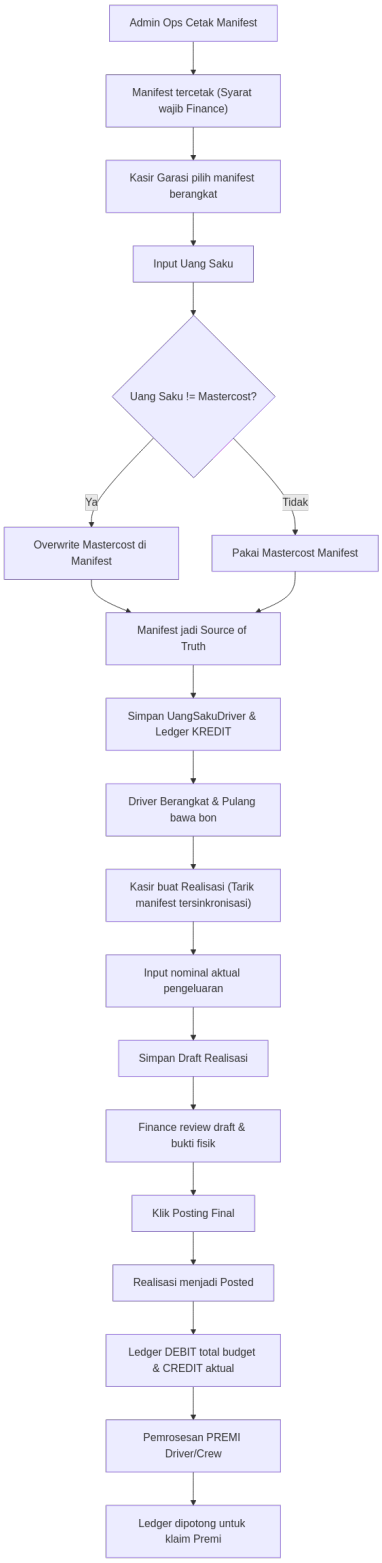
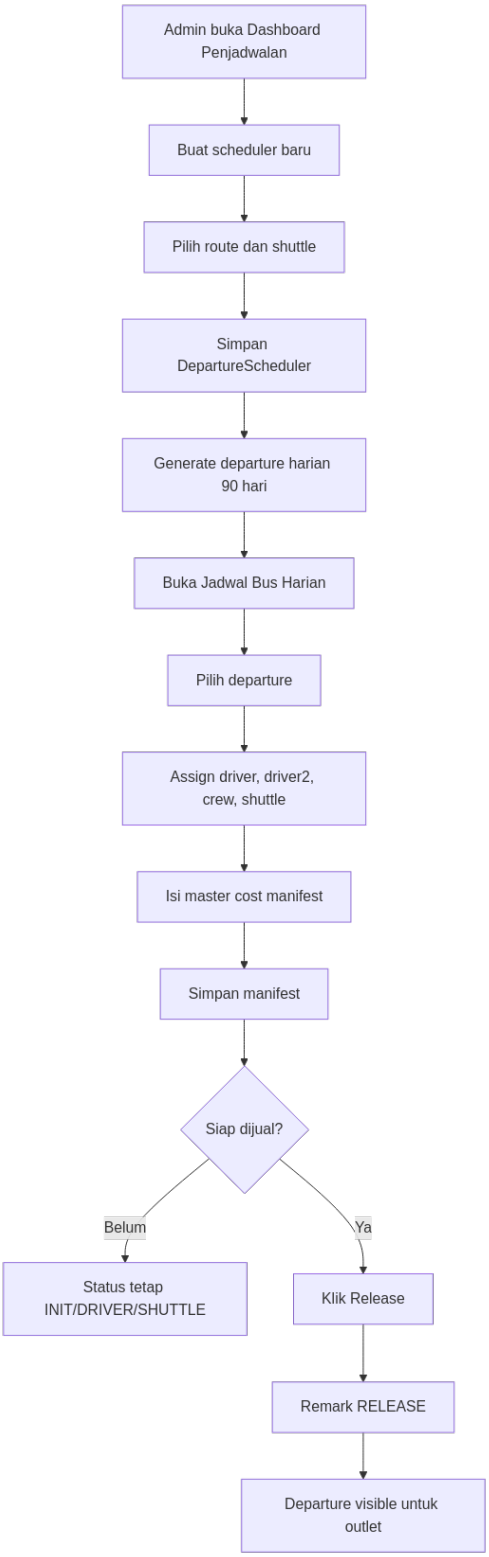
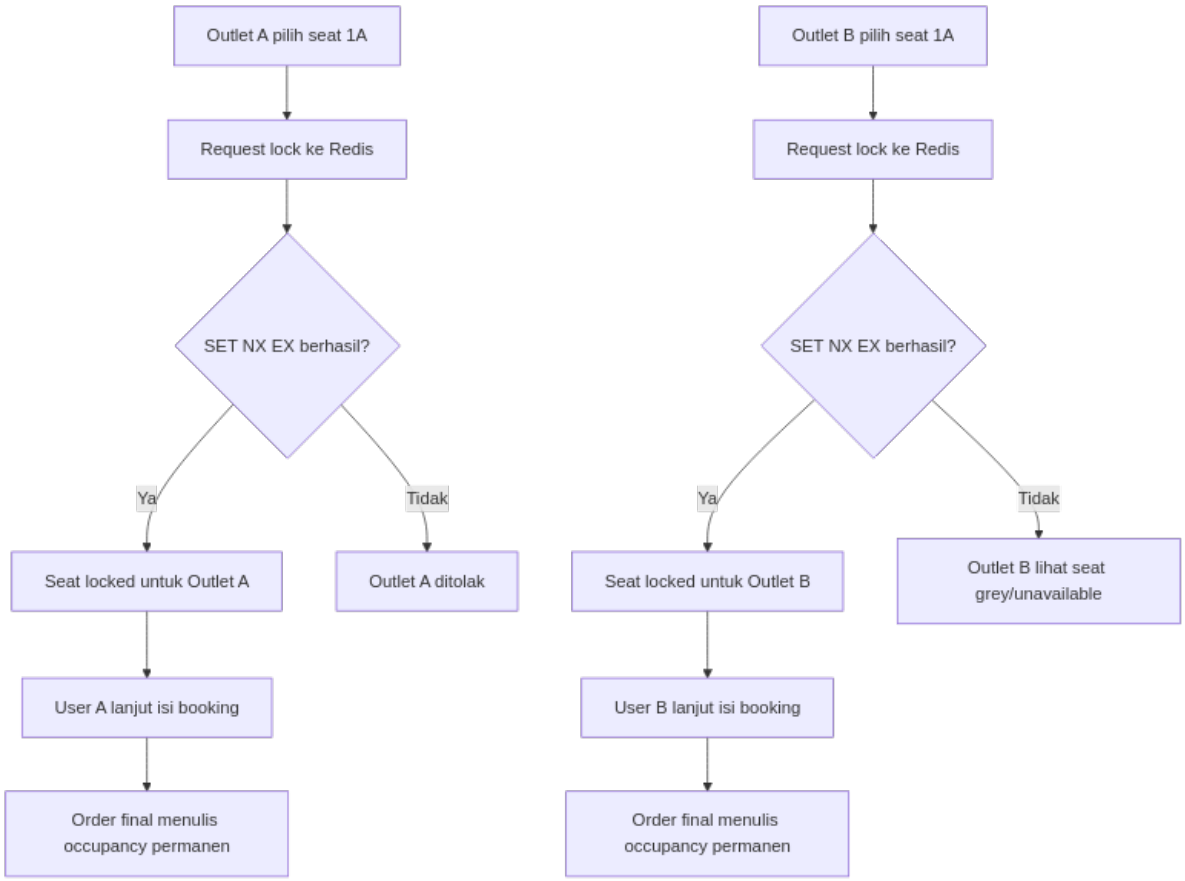


Figure 1: Di sinilah Uang Saku meng-
overwrite Manifest, Realisasi menarik manifest, Posting tereksekusi, dilanjut Premi.

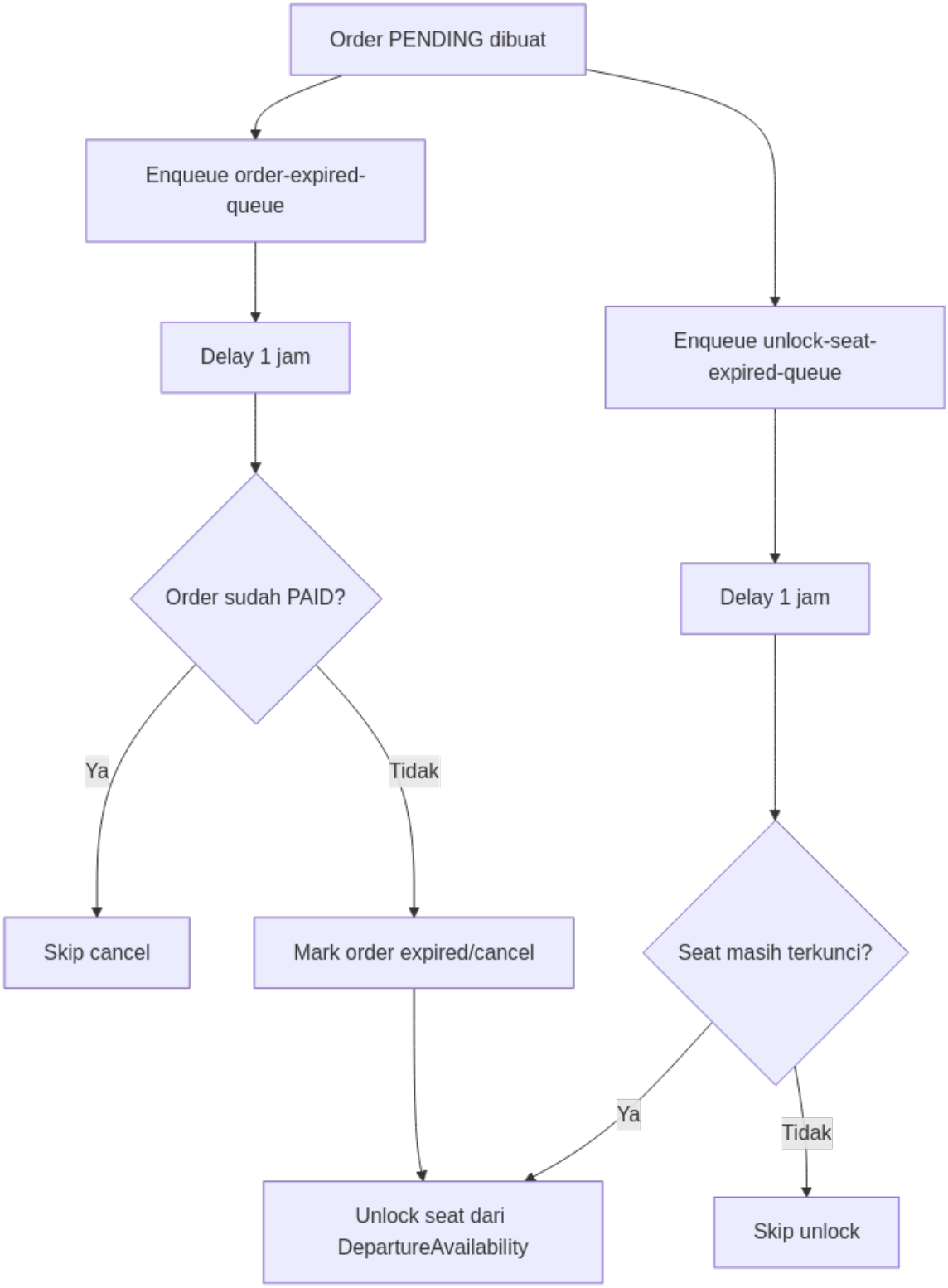
4.d Schedule Preparation Flow



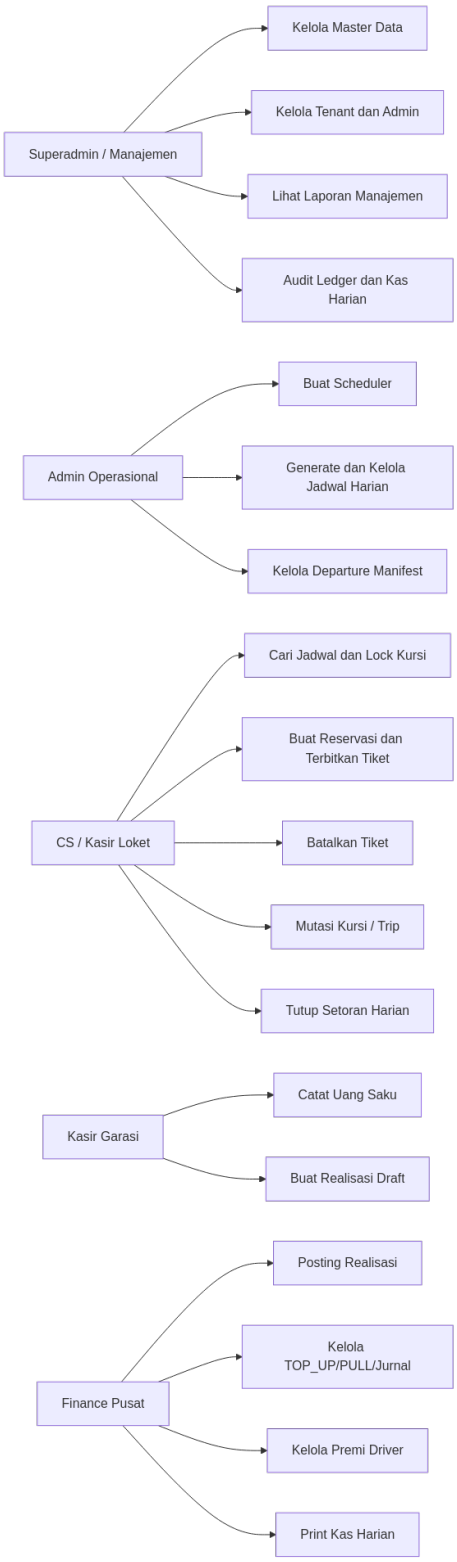
4.e Seat Locking Race Condition Prevention



4.f Order Expiry BullMQ Worker Flow



5. Use Case Diagram & Deskripsi



UC13: Catat Uang Saku (Overwriting logic)

- **Aktor**: Kasir Garasi
- **Prekondisi**: Manifest keberangkatan TELAH DICETAK dan dieksekusi Admin Opera-

sional.

- **Skenario Utama**: Kasir memasukkan jumlah *BBM, Tol*, bila berbeda dengan Mastercost asli di jadwal rutin, nilai ini OVERWRITE nominal di tabel manifest. Manifest dikunci menjadi *Source of Truth.* Ledger Kredit tercatat.

UC14: Buat Realisasi Draft

- **Aktor**: Kasir Garasi
- **Prekondisi**: Supir tiba membuahkan bon fisik.
- **Skenario Utama**: Kasir melengkapi rincian. **Acuan Realisasi sepenuhnya berasal dari Manifest yang sudah ter-overwrite dari Uang Saku tadi**. Draft diajukan ke Finance.

UC17: Kelola Premi Driver/Crew (Baru Ditambahkan)

- **Aktor**: Finance Pusat
- **Prekondisi**: Supir berhasil merealisasikan perjalanannya.
- **Skenario Utama**: Finance memverifikasi daftar *claim flag.* Rate dinilai berdasarkan role (Driver / Crew). Uang potongan ditarik dari Ledger. Status ditutup agar tidak diklaim dua kali.